```
# Conditions that come from T alone (no second copy). Inputs: T and the
Cartan/weights derived from it (script 26).
import numpy as np, io, contextlib, itertools
with contextlib.redirect_stdout(io.StringIO()):
exec(open('26_roots_from_form_only.py').read())
rng=np.random.RandomState(3); co=rng.randn(len(Hs)); V=[];F=[]
for s in range(3):
    Hg=sum(c*H[s] for c,H in zip(co,Hs)); w,VV=np.linalg.eig(Hg)
    Fs=np.array([[ (v.conj()@H[s]@v/(v.conj()@v)).imag for H in Hs] for v in VV.T]);
V.append(VV);F.append(Fs)
Tw=np.einsum('abc,ai,bj,ck->ijk',T,V[0],V[1],V[2])
nz=np.abs(Tw)>1e-9
S=F[0][:,None,None,:]+F[1][None,:,None,:]+F[2][None,None,:,:];
Z=np.linalg.norm(S,axis=3)<1e-6
print("(a) T in the weight basis: nonzero entries =",int(nz.sum())," | weight triples
with w_v+w_s+w_c=0:",int(Z.sum()))
print("    every nonzero entry has zero weight-sum:",bool((nz<=Z).all())," | every
zero-sum triple is nonzero:",bool((Z<=nz).all()))
# (b) count zero-sum triples of block elements for each of the six bijections (uses
weights only)
Ch=np.linalg.cholesky(sc*Kinv); Wl=[f@Ch for f in F]
def blk(X,Y): return np.hstack([np.repeat(Wl[X],8,0),np.tile(Wl[Y],(8,1))])
print("\n(b) triples (one element from each of the three blocks) summing to zero, per
bijection:")
for p in itertools.permutations(range(3)):
    b=[blk(X,p[X]) for X in range(3)]
    tot=b[0][:,None,None,:]+b[1][None,:,None,:]+b[2][None,None,:,:]
    print("   ","".join("vsc"[X]+"vsc"[p[X]]+" " for X in
range(3)),":",int((np.linalg.norm(tot,axis=3)<1e-6).sum()))
# (c) are argument permutations of T symmetries of T (up to conjugation of each
argument)?
Cj=np.diag([1]+[-1]*7).astype(float); I8=np.eye(8)
print("\n(c) argument permutation sigma of T realised as T(P1 x,P2 y,P3 z) with each
P in {identity, conjugation}:")
for sg in itertools.permutations(range(3)):
    Ts=T.transpose(sg); found=[]
    for P in itertools.product([I8,Cj],repeat=3):
        Tp=np.einsum('abc,ai,bj,ck->ijk',T,*P)
        if np.abs(Tp-Ts).max()<1e-12: found.append(["I" if np.allclose(m,I8) else
"conj" for m in P])
        elif np.abs(Tp+Ts).max()<1e-12: found.append(["I" if np.allclose(m,I8) else
"conj" for m in P]+["(sign -1)"])
    print("   sigma",sg,":",found if found else "no realisation in this search")
```